Multi-Container Runtime with Kernel Memory Monitor

Chatresh Ramasai Gudi(PES1UG24CS918)
Raiesh Giriyapanavar(PES1UG24CS924)

---

1 Introduction

This project implements a custom container runtime environment featuring a user-space supervisor and a specialized Linux kernel module for real-time memory monitoring and enforcement. The system provides process isolation via Linux namespaces and manages container lifecycles through a centralized engine.

2 Build and Setup Instructions

2.1 Prerequisites

The implementation is designed for Ubuntu 22.04/24.04 environments. It requires kernel headers for

```
sudo apt update
sudo apt install -y build-essential linux-headers-$(
uname -r)
```

module compilation and Secure Boot must be disabled to load the custom kernel module.

2.2 Build and Load

```
# Build all components make -C
boilerplate

# Load the kernel monitor sudo insmod
boilerplate/monitor.ko
```

The project uses a structured Makefile to compile the supervisor, kernel monitor, and test workloads.

3 System Architecture & Demo Demos

3.1 Multi-Container Supervision

The supervisor process creates isolated environments using clone() with CLONE_NEWPID, CLONE_NEWNS, CLONE_NEWUTS, and CLONE_NEWNET ags.

```
raiesh@raiesh-VirtualBox:~/OS-Jackfruit/boilerplate$ make
gcc -O2 -Wall -Wextra -o engine engine.c -lpthread
gcc -O2 -Wall -static -o memory_hog memory_hog.c
gcc -O2 -Wall -static -o cpu_hog cpu_hog.c
gcc -O2 -Wall -static -o io_pulse io_pulse.c
make -C /lib/modules/6.17.0-20-generic/build M=/home/raiesh/OS-Jackfruit/boilerplate modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-20-generic'
make[2]: Entering directory '/home/raiesh/OS-Jackfruit/boilerplate'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  You are using:           gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  CC [M]  monitor.o
  MODPOST Module.symvers
  CC [M]  monitor.mod.o
  LD [M]  monitor.ko
  BTF [M] monitor.ko
Skipping BTF generation for monitor.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/raiesh/OS-Jackfruit/boilerplate'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-20-generic'
```

Figure 1: Demo 1: Concurrent execution of 'alpha' and 'beta' containers.

3.2 Metadata and State Tracking

The supervisor tracks the state of every container, including its PID, memory limits, and scheduling priority (nice value).

```
raiesh@raiesh-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
[sudo] password for raiesh:
tracked containers
id      pid     state   started_at      soft_mib        hard_mib        exit
beta    9920    running 2026-04-22 13:04:16     40      64      0/0
alpha   9885    running 2026-04-22 13:03:48     40      64      0/0
```

Figure 2: Demo 2: Metadata tracking via 'ps' command output.

3.3 Logging Pipeline

Logs are managed via a bounded-bu er producer/consumer model. Producers read from container pipes and a single logger thread ushes data to per-container les.

```
raiesh@raiesh-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine logs alpha
logs for alpha
[supervisor] started pid=9885 rootfs=./rootfs-alpha command=/bin/sh soft=41943040 hard=67108864 nice=0
raiesh@raiesh-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine logs beta
logs for beta
[supervisor] started pid=9920 rootfs=./rootfs-beta command=/bin/sh soft=41943040 hard=67108864 nice=0
```

Figure 3: Demo 3: Bounded-bu er logging contents for separate containers.

```
raiesh@raiesh-VirtualBox:~/OS-Jackfruit/boilerplate$ ./cpu_hog
cpu_hog alive elapsed=1 accumulator=7599819152365339343
cpu_hog alive elapsed=2 accumulator=1699396606794199378
cpu_hog alive elapsed=3 accumulator=3421429043610886688
cpu_hog alive elapsed=4 accumulator=8563933929715771403
cpu_hog alive elapsed=5 accumulator=11059455012088195326
cpu_hog alive elapsed=6 accumulator=18288386422116255765
cpu_hog alive elapsed=7 accumulator=2034393935029890564
cpu_hog alive elapsed=8 accumulator=15155307866675224077
cpu_hog alive elapsed=9 accumulator=10784897008375183026
cpu_hog done duration=10 accumulator=16372215635578547241
```

Figure 4: Demo 4: CLI interaction and IPC control channel via UNIX domain sockets.

3.4 Memory Limit Enforcement

The kernel module monitors RSS (Resident Set Size). It triggers a KERN_WARNING for soft limits and sends SIGKILL for hard limit violations.

```
raiesh@raiesh-VirtualBox:~/OS_Jackfruit/boilerplate$ dmesg | grep container_monitor
[ 3297.083247] [container_monitor] Registering container=alpha pid=9138 soft=50331648 hard=83886080
[ 3303.565366] [container_monitor] Registering container=beta pid=9153 soft=67108864 hard=100663296
[ 4292.331922] [container_monitor] Registering container=alpha pid=9885 soft=41943040 hard=67108864
[ 4320.826867] [container_monitor] Registering container=beta pid=9920 soft=41943040 hard=67108864
[ 4873.091308] [container_monitor] Registering container=alpha pid=10057 soft=50331648 hard=83886080
[ 4899.695331] [container_monitor] Registering container=beta pid=10072 soft=67108864 hard=100663296
[ 6252.622998] [container_monitor] Registering container=alpha pid=10402 soft=50331648 hard=83886080
[ 6259.850635] [container_monitor] Registering container=beta pid=10407 soft=67108864 hard=100663296
[ 6422.704654] [container_monitor] Module unloaded.
[ 7523.243267] [container_monitor] Module loaded. Device: /dev/container_monitor
[ 7663.039177] [container_monitor] Registering container=alpha pid=12670 soft=41943040 hard=67108864
[ 7706.740595] [container_monitor] Registering container=beta pid=12703 soft=41943040 hard=67108864
[ 7854.808634] [container_monitor] Registering container=gamma pid=12764 soft=41943040 hard=67108864
[ 7882.976960] [container_monitor] Registering container=mem1 pid=12784 soft=8388608 hard=25165824
[ 7884.835348] [container_monitor] SOFT LIMIT container=mem1 pid=12784 rss=8986624 limit=8388608
[ 7888.934508] [container_monitor] HARD LIMIT container=mem1 pid=12784 rss=25763840 limit=25165824
[ 7962.630924] [container_monitor] Registering container=cpu1 pid=12812 soft=41943040 hard=67108864
[ 7989.652755] [container_monitor] Registering container=cpu2 pid=12839 soft=41943040 hard=67108864
```

Figure 5: Demo 5: Figure 6: Demo 6: Kernel logs showing Soft-Limit warning emission. Hard-Limit enforcement and resulting 'hard_limit_killed' state.

3.5 Scheduling and Teardown

Scheduling is managed via CFS (Completely Fair Scheduler) weights derived from nice values.

```
raiesh@raiesh-VirtualBox:~/OS_Jackfruit/boilerplate$ sudo ./engine ps
cpu2    12839   running
cpu1    12812   running
mem1    12784   exited
gamma   12764   exited
beta    12703   running
alpha   12670   running
```

Figure 7: Demo 7: Scheduling experiment comparing nice=0 vs nice=10 workloads.

```
raiesh@raiesh-VirtualBox:~/OS_Jackfruit/boilerplate$ sudo ./engine start cpu1 ./rootfs-cpu1 "/cpu_hog 15" --nice 0
Container started
raiesh@raiesh-VirtualBox:~/OS_Jackfruit/boilerplate$ sudo ./engine start cpu2 ./rootfs-cpu2 "/cpu_hog 15" --nice 10
Container started
```

```
raiesh@raiesh-VirtualBox:~/OS_Jackfruit/boilerplate$ sudo ./engine logs cpu1
cpu_hog alive elapsed=1 accumulator=1914556318973563578
cpu_hog alive elapsed=2 accumulator=7857074313416860710
cpu_hog alive elapsed=3 accumulator=3546994770694941647
cpu_hog alive elapsed=4 accumulator=5389061045105652623
cpu_hog alive elapsed=5 accumul
raiesh@raiesh-VirtualBox:~/OS_Jackfruit/boilerplate$ sudo ./engine logs cpu2
cpu_hog alive elapsed=1 accumulator=12217336799086068511
cpu_hog alive elapsed=2 accumulator=10506865500450932922
cpu_hog alive elapsed=3 accumulator=14239396131153841599
cpu_hog alive elapsed=4 accumulator=14415095576852766355
cpu_hog alive elapsed=5 acc
```

```
raiesh@raiesh-VirtualBox:~/OS_Jackfruit$ sudo make -C boilerplate clean
make: Entering directory '/home/raiesh/OS_Jackfruit/boilerplate'
if [ -d "/lib/modules/6.17.0-20-generic/build" ]; then make -C /lib/modules/6.17.0-20-generic/build M=/home/raiesh/OS_Jackfruit/boilerpl
ate clean; fi
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-20-generic'
make[2]: Entering directory '/home/raiesh/OS_Jackfruit/boilerplate'
make[2]: Leaving directory '/home/raiesh/OS_Jackfruit/boilerplate'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-20-generic'
rm -f engine memory_hog cpu_hog io_pulse *.o *.mod *.mod.c *.symvers *.order
rm -f *.log
rm -rf logs
rm -f /tmp/mini_runtime.sock
make: Leaving directory '/home/raiesh/OS_Jackfruit/boilerplate'
```

Figure 8: Demo 8: Clean teardown and supervisor exit sequence.

4 Engineering Analysis

4.1 Memory Management Design

RSS was chosen over Virtual Memory size because it re ects actual physical RAM consumption. Enforcement is handled in kernel-space to eliminate the race window between monitoring and intervention.

4.2 Scheduler Behavior

Using the formula weight $\approx 1024/(1.25^{\text{nice}})$, a container with nice 0 receives roughly 9x more CPU share than nice 10 under contention. In our tests, both finished in 15 seconds real-time, but their internal throughput (loop iterations) differed based on CPU availability.

5 Conclusion

The system successfully demonstrates the fundamentals of containerization: namespace-based isolation, metadata-driven lifecycle management, and kernel-level resource enforcement.